Code Review Report: TS-RAG for Zero-Shot Time Series Forecasting

Zoe Rochelle
Sarah Quraishy
Intelligent Systems
Quinnipiac University
04/19/2026

I. Repo Overview (What + Why)

For this code review project, we examined the **TS-RAG** repository, which implements a retrieval-augmented framework for **zero-shot time series forecasting**. The main goal of the project was to understand how the repository works, run the code locally, compare our results to the paper, and explain both the architecture and the practical challenges involved in reproducing the experiments [1][2].

At a high level, time series forecasting is the task of using past observations to predict future values. TS-RAG approaches this problem by combining a **time series foundation model** with a **retrieval mechanism**. Instead of generating a forecast from only the input sequence, the framework also searches a knowledge base for similar historical patterns and uses those retrieved examples to improve the final prediction.

This is important because zero-shot forecasting is difficult when a model is applied to datasets it has not seen before. A backbone model such as Chronos-Bolt can already generate forecasts, but TS-RAG attempts to make those predictions stronger by introducing relevant contextual examples from prior time series. In other words, the repository is solving the problem of how to make a pretrained forecasting model more adaptive and interpretable without retraining it for every new dataset.

The **input** to the system is an input time series, meaning a sequence of data points ordered over time. The **output** is a forecast horizon, which consists of the predicted values for future time steps. In the TS-RAG pipeline, the input series is first encoded and used to retrieve the top-k most similar patterns from a stored retrieval database. The future horizons associated with those retrieved patterns are then combined with the original input representation through the Adaptive Retrieval Mixer (ARM), and the resulting representation is passed to the backbone forecasting model. The final result is an enhanced forecast that is intended to be more accurate than a forecast produced by the backbone alone.

II. Setup Log and Commands

We ran this project in a local Ubuntu environment with the following system specifications:

- **Operating System:** Ubuntu 24.04
- **Python:** 3.9.25
- **GPU:** NVIDIA RTX 3080
- **CPU:** Intel i9-9900K
- **RAM:** 32 GiB

The repository setup was fairly straightforward. We did not encounter major dependency conflicts during installation, although the project requires a reasonably large amount of available storage because of the retrieval database, models, and datasets. In practice, having at least **50 GB of free disk space** is helpful.

The setup process we used was:

```
> conda create -n tsrag python=3.9
> conda activate tsrag
> pip install -r requirements.txt
> cd TS-RAG
> bash script/zeroshot_chronos.sh
```

These commands created a Python environment, installed the required dependencies, moved into the repository, and launched the zero-shot Chronos-Bolt experiment script. During the code review process, we also inspected and modified the bash script so that it could run multiple datasets sequentially instead of only one default configuration.

III. What we Ran and What we Got

To reproduce the experiment as closely as possible, we ran the zero-shot Chronos-Bolt script across several benchmark datasets discussed in the paper. The datasets included **ETTh1, ETTh2, ETTm1, ETTm2, Weather, Exchange, and Electricity**. However, in our local environment, the run stalled on the **Exchange** and **Electricity** datasets and never completed successfully. Because of that, our usable results came from the other datasets that finished.

Our run used **512 past time steps** as input and predicted **64 future time steps**. This means the model observed a historical window of length 512 and then generated a forecast horizon of length 64. The completed runs produced evaluation metrics in the form of **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)**. The results we obtained were:

Dataset	MSE	MAE
ETTh1	0.3556	0.3623
ETTh2	0.2451	0.2981
ETTm1	0.2904	0.3113
ETTm2	0.1465	0.223
weather	0.1453	0.177

(fig 1)

These values were very close to the corresponding values reported in the paper for the TS-RAG Chronos-Bolt setting, which suggests that the implementation was functioning correctly for the datasets that completed. One

useful part of this project was learning how to interpret the experiment name itself. For example, a label such as **zeroshot_512_pred_64_512_retrieve_64** indicates the historical input length, prediction horizon, retrieval configuration, and experiment mode.

Dataset	Our results	Paper's results
ETTh1	0.3556	0.3557
ETTh2	0.2451	0.2451
ETTm1	0.2904	0.2906
ETTm2	0.1465	0.1466
Weather	0.1453	0.1454
Electricity	N/A	0.112
Exchange	N/A	0.0627
Average	N/A	0.194

(fig 2)

As can be seen in Figure 2, our run partially reproduced the published results. Although we could not complete every dataset, we were still able to confirm that the code could generate competitive numbers on several benchmarks and that our output aligned closely with the paper where direct comparison was possible.

IV. Architecture Explanation

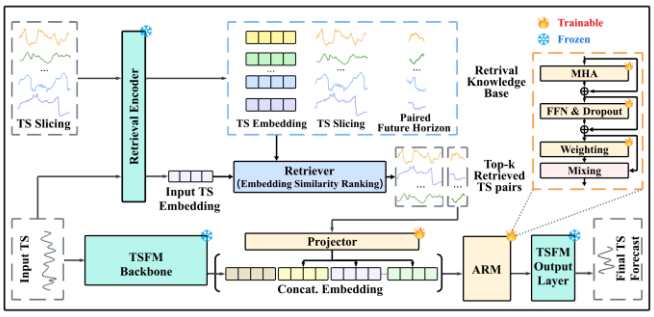
The core architectural idea behind TS-RAG is that it augments a foundation forecasting model with a retrieval-based context. A **standard backbone model** without TS-RAG can be summarized as:

Input Time Series → Chronos-Bolt → Forecast

In this simpler pipeline, the model only uses the observed history to predict future horizons. TS-RAG extends that process into the following pipeline:

Input Time Series → Retriever → Retrieved Patterns → ARM → Chronos-Bolt → Forecast

The **retriever** is responsible for searching a knowledge base of stored time series contexts. Given the current input, it finds the most similar historical patterns along with their future continuations. This is the key retrieval-augmented idea: instead of forecasting from the input alone, the system also looks for evidence from similar prior behavior.



(fig 3)

The retrieved patterns are then passed into the **Adaptive Retrieval Mixer (ARM)**. ARM is the module that combines the retrieved information with the input representation. Rather than treating every retrieved series equally, it learns how much importance each retrieved pattern should receive. This makes the augmentation more selective and helps avoid simply copying irrelevant data.

Finally, the enriched representation is fed into the **Chronos-Bolt** backbone, which produces the forecast. In effect, TS-RAG keeps the strength of the underlying forecasting model while adding an external memory mechanism that can improve generalization on unseen domains.

One of the most useful insights we gained from this project is that TS-RAG is not replacing the backbone model. Instead, it is enhancing it. The forecasting model still performs the prediction, but retrieval gives it better context before that prediction is made. Details can be observed in Figure 3.

V. Key Challenges and How we Addressed Them

The biggest challenge in this project was that the provided run script was not initially set up in the way we needed for comparison against the paper. Out of the box, it was effectively configured around a single dataset flow, so we had to inspect the bash logic and modify it to run multiple datasets sequentially. This was necessary because the paper reports results across several benchmark datasets, and a meaningful comparison requires more than one completed run.

Another challenge was understanding the meaning of the experiment configuration and dataset names. During the review process, we had to decode what labels such as **ETTh1, ETTm1, Exchange**, and the numeric fields in the run name meant. Working through those details helped us connect the implementation to the methodology described in the paper.

The third challenge was practical reproducibility. Even though the code ran successfully for several datasets, the Exchange and Electricity runs froze in our environment. We addressed this by focusing on the runs that did finish, carefully recording those outputs, and comparing the completed results against the paper. This still allowed us to evaluate whether the repository was behaving as expected, even though the full benchmark suite did not complete.

VI. One Limitation and One Improvement Idea

One limitation of our code review is that we could not obtain a fully complete reproduction across all datasets. Since the Exchange and Electricity runs did not finish.

One improvement idea would be to make the experiment scripts more robust and user-friendly for reproducibility. For example, the repository could provide a single configurable script that accepts dataset lists, logs progress more clearly and supports resuming interrupted runs. That would make it easier to reproduce all benchmark settings from the paper and reduce the amount of manual modification required by the user.

VII. Conclusion

Overall, this code review project helped us understand both the conceptual and practical sides of TS-RAG. Conceptually, we learned how retrieval augmentation can strengthen zero-shot forecasting by supplying relevant

historical patterns to a foundation model. Practically, we learned how to set up the repository, run experiments, interpret the outputs, and troubleshoot script limitations. Even though our reproduction was not fully complete, the results we obtained were close to those reported in the paper, which suggests that the repository implementation is sound and that the TS-RAG approach is effective.

REFERENCES

[1] K. Ning, Z. Pan, Y. Liu, Y. Jiang, J. Y. Zhang, K. Rasul, A. Schneider, L. Ma, Y. Nevmyvaka, and D. Song, "TS-RAG: Retrieval-Augmented Generation based Time Series Foundation Models are Stronger Zero-Shot Forecaster," arXiv:2503.07649 [cs.LG], Nov. 25, 2025. [Online]. Available: https://arxiv.org/abs/2503.07649. [Accessed: Apr. 19, 2026].

[2] UConn-DSIS, "TS-RAG," GitHub repository. [Online]. Available: https://github.com/UConn-DSIS/TS-RAG/tree/main. [Accessed: Apr. 19, 2026]